BUILD YOUR FIRST GENERATIVE AI DATA EXTRACTION APPLICATION

STUDENT NOTES — 60 MINUTES

Make AI Read • Understand • Extract • Structure

Python • LangChain • Structured Output • Pydantic

1. Workshop Goal

Convert normal human language into clean, structured, application-ready data.

```
UNSTRUCTURED TEXT
        ↓
   CHAT MODEL
        +
 PYDANTIC SCHEMA
        ↓
with_structured_output()
        ↓
 PYDANTIC OBJECT
        ↓
FIELDS / DICT / JSON
```

MASTER IDEA: Natural Language + AI Model + Pydantic Schema → Structured Data

2. Unstructured Text vs Structured Data

Unstructured text: Information inside normal sentences or paragraphs.

```
Ravi is 22 years old and learning Python.
```

Structured data: Information separated into clear fields and values.

```
name   = Ravi
age    = 22
course = Python
```

IMPORTANT: Plain text is excellent for humans. Structured data is often better for applications.

3. Structured Output and Schema

Structured Output: Model output that follows a predefined structure instead of free-form text.

Schema: Defines expected fields, meaning and data types.

```
Student
 ├── name   → str
 ├── age    → int
 └── course → str
```

- A field represents one piece of data.
- Use suitable data types: str, int, float, bool, list[str].

IMPORTANT: Schema = What structure our application expects.

4. Pydantic Basics

Pydantic helps us define structured Python data models using fields and data types.

```
from pydantic import BaseModel
class Student(BaseModel):
    name: str
    age: int
    course: str
student = Student(
    name="Ravi",
    age=22,
    course="Python"
)
print(student)
print(student.name)
print(student.age)
print(student.course)
```

IMPORTANT: BaseModel is commonly used to create a Pydantic data model.

5. Pydantic Object → Dictionary / JSON

```
data = student.model_dump()
print(data)
json_data = student.model_dump_json()
print(json_data)
```

model_dump() → Python dictionary

model_dump_json() → JSON string

6. First AI Structured Output Program

Let the LLM read natural language and return a Student object.

```
import os
from langchain_openai import ChatOpenAI
from pydantic import BaseModel
class Student(BaseModel):
```

```
    name: str
    age: int
    course: str
llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
structured_llm = llm.with_structured_output(Student)
result = structured_llm.invoke(
    "Ravi is 22 years old and learning Python."
)
print(result)
print(type(result))
print(result.name)
print(result.age)
print(result.course)
```

IMPORTANT: llm.invoke() normally gives an AIMessage. Structured output with a Pydantic schema gives a Pydantic object when successful.

7. Key API: with_structured_output()

```
structured_llm = llm.with_structured_output(Student)
```

Meaning: Create a model wrapper configured to return output matching the Student schema.

with_structured_output() creates a structured-output model wrapper; it does not permanently modify the original llm object.

8. Required and Optional Fields

```
name: str
email: str | None = None
```

Required field: A value is expected.

Optional field: Value may be unavailable; None is allowed.

GOLDEN RULE: If information is missing, do not invent it. Use None for optional fields.

9. Field() Descriptions

```
from pydantic import BaseModel, Field
class Employee(BaseModel):
    name: str = Field(
        description="Name of the employee"
    )
    experience: int = Field(
```

```
        description="Number of years of professional experience"
    )
```

Good field names + correct data types + clear descriptions = clearer schema.

10. Final Generative AI Data Extraction Application

```
import os
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field
class Candidate(BaseModel):
    name: str = Field(
        description="Name of the person"
    )
    profession: str = Field(
        description="Profession or job role"
    )
    city: str | None = Field(
        default=None,
        description="City where the person lives"
    )
    experience: int | None = Field(
        default=None,
        description="Years of professional experience"
    )
    skills: list[str] = Field(
        description="Professional skills mentioned in the text"
    )
    email: str | None = Field(
        default=None,
        description="Email address if mentioned"
    )
llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
structured_llm = llm.with_structured_output(Candidate)
text = input("Enter candidate information: ")
instruction = f"""
Extract candidate information from the text below.
Rules:
- Use only information available in the text.
- Do not invent missing information.
- If optional information is unavailable, return None.
Text:
{text}
"""
result = structured_llm.invoke(instruction)
print("\nEXTRACTED INFORMATION")
print("Name       :", result.name)
print("Profession :", result.profession)
```

```
print("City       :", result.city)
print("Experience :", result.experience)
print("Skills     :", result.skills)
print("Email      :", result.email)
print("\nPYTHON DICTIONARY")
print(result.model_dump())
print("\nJSON STRING")
print(result.model_dump_json())
```

11. Sample Input

```
Ravi is a Python developer from Hyderabad.
He has 4 years of professional experience.
His skills include Python, Django, FastAPI and SQL.
His email is ravi@example.com.
```

Expected fields: name, profession, city, experience, skills and email.

12. Reliability Rules

- Use only information available in the provided text.
- Do not invent missing information.
- Use optional fields for data that may not exist.
- Choose correct data types for future application use.
- Use clear field names and descriptions.
- Start with a small schema; increase fields only when needed.

IMPORTANT: Structured output gives predictable shape and types, but it does not automatically guarantee that every real-world fact is correct.

13. Common Mistakes

Mistake 1: Making missing information required.

Mistake 2: Asking AI to guess or complete every field somehow.

Mistake 3: Using str for everything when int/float/bool/list is more suitable.

Mistake 4: Using unclear field names such as x, y, z.

Mistake 5: Creating a huge schema at the beginning.

14. Real-World Applications

- Resume Extractor — name, skills, experience, education, email
- Customer Extractor — name, city, phone, email
- Product Extractor — name, category, price, availability
- Student Extractor — name, age, course, email

- Support Ticket Extractor — customer, problem, product, priority

15. Connection to GenAI and Agentic AI

```
GENERATIVE AI
     ↓
READ + UNDERSTAND + EXTRACT
     ↓
STRUCTURED DATA
     ↓
FUTURE AGENT
     ↓
DECISION / ACTION
```

Structured data can become useful input for later Agentic AI decisions and actions.

16. Quick Revision

- Unstructured text = information inside free-form sentences or paragraphs.
- Structured output = output according to a predefined schema.
- Schema = expected fields, meaning and data types.
- Pydantic BaseModel = define structured Python models.
- Optional field = field may contain None.
- Field() = add descriptions/defaults/validation information.
- with_structured_output() = configure the model to return schema-based output.
- model_dump() = Pydantic object to Python dictionary.
- model_dump_json() = Pydantic object to JSON string.

17. Must-Remember Syntax

```
class Student(BaseModel):
    name: str
    age: int
    email: str | None = None
structured_llm = llm.with_structured_output(Student)
result = structured_llm.invoke(text)
print(result.name)
print(result.model_dump())
print(result.model_dump_json())
```

18. Student Practice

Exercise 1 — Course Extractor

```
class Course(BaseModel):
    name: str
    trainer: str
    duration: int
```

Input: Python is taught by Durga and the course duration is 60 days.

Exercise 2 — Resume Extractor

```
class Resume(BaseModel):
    name: str
    technology: str
    experience: int | None = None
```

Input: Suresh is a Java developer with 5 years of experience.

Exercise 3 — Customer Extractor

```
class Customer(BaseModel):
    name: str
    city: str | None = None
    phone: str | None = None
```

Input: Ravi lives in Hyderabad. Expected phone value: None.